

LeetCode 2035: Partition Array Into Two Arrays to Minimize Sum Difference

Complete Notes & Documentation (Based on Our Discussion)

1. Problem Understanding

Problem Statement

Given an array where $\text{nums.length} = 2n$, divide the array into two arrays such that:

- Both arrays contain exactly n elements.
- The absolute difference between their sums is minimized.

Return: $|\text{sum}(\text{array1}) - \text{sum}(\text{array2})|$

Example

```
nums = [2, -1, 0, 4, -2, -9]
```

Possible partition:

- **Array1:** $[2, 4, -9] \rightarrow \text{sum}_1 = -3$
- **Array2:** $[-1, 0, -2] \rightarrow \text{sum}_2 = -3$
- **Difference:** $|-3 - (-3)| = 0$

Answer: 0

2. Why Normal Subset Sum DP Fails?

Positive Number DP

For positive numbers, we traditionally use a table structure like $\text{dp}[\text{index}][\text{sum}]$.

Example: $\text{nums} = [1, 2, 3] \rightarrow$ Possible sums: $0, 1, 2, 3, 4, 5, 6$ (All positive).

Problem 1: Negative Numbers

Example: $\text{nums} = [-5, 2, -3] \rightarrow$ Possible sums: $-8, -5, -3, -1, 2, \dots$

Now we would need to reference states like $\text{dp}[\text{index}][-5]$, which is impossible because array indices cannot be negative.

Problem 2: Huge Constraints

- $\text{nums.length} \leq 30$
- $\text{nums}[i] \leq 10^7$

- Maximum total sum: $30 \times 10^7 = 3 \times 10^8$

This would require a DP size of `dp[30][300000000]`, which is entirely impossible due to memory limits.

Conclusion

Classic subset-sum Dynamic Programming cannot be used. We need an alternative approach.

3. Key Observation

Array length: $2n \leq 30$. Therefore, $n \leq 15$.

This bound ($n \leq 15$) is the perfect range for a **Meet In The Middle** approach.

4. Mathematical Transformation

We need to minimize:

$$\text{difference} = |S_1 - S_2|$$

We know that:

$$S_2 = \text{totalSum} - S_1$$

Substituting S_2 into the difference formula:

$$\text{difference} = |S_1 - (\text{totalSum} - S_1)| = |2 \times S_1 - \text{totalSum}|$$

Important Insight

We only need to track and find the subset sum S_1 , because S_2 is automatically determined by the remaining elements.

5. Meet In The Middle

Split the array into two halves.

Example: `nums = [2, -1, 0, 4, -2, -9]`

```
left  = [2, -1, 0]
right = [4, -2, -9]
```

Instead of generating 2^{30} total subsets directly, we generate 2^{15} subsets on each side independently.

6. Core Idea

- Store `left[count]` = All subset sums of the left half having exactly `count` elements.
- Store `right[count]` = All subset sums of the right half having exactly `count` elements.

Example: `left[2]` means all subset sums formed using exactly 2 elements from the left subarray.

7. Bitmasking Fundamentals

What is a Mask?

For a collection like `[2, -1, 0]`, each element has a choice to either be taken or skipped (Take / Don't Take). Total subsets = $2^3 = 8$.

Masks represent these combinations in binary format:

```
000, 001, 010, 011, 100, 101, 110, 111
```

Meaning: `1` = Take element, `0` = Skip element.

Example: `mask = 101` (binary) means:

- Take index 0
- Skip index 1
- Take index 2

Resulting subset: `{2, 0}`

8. Understanding This Line: `(mask & (1 << i))`

What does `1 << i` do?

```
1 << 0 = 001
1 << 1 = 010
1 << 2 = 100
```

Why AND?

Bit A	Bit B	A & B
1	1	1
1	0	0
0	1	0

Bit A	Bit B	A & B
0	0	0

Example: Given `mask = 101`

Check bit 0:

```

101
& 001
-----
001 (Non-zero → Bit 0 is ON)

```

Check bit 1:

```

101
& 010
-----
000 (Zero → Bit 1 is OFF)

```

Intuition

Whenever you see `(mask & (1 << i))`, read it as: "Is the i-th element included in this subset?"

9. Generating Left and Right Subsets Simultaneously

Code block snippet:

```

leftSum += nums[i];
rightSum += nums[i+n];

```

Why? Suppose `left = [2, -1, 0]` and `right = [4, -2, -9]`.

A `mask = 101` means take position 0 and position 2. Simultaneously:

- For left: $2 + 0 = 2$
- For right: $4 + (-9) = -5$

Both are generated together efficiently within the same loop sequence.

10. Why Store By Count?

If $n = 3$, we need exactly `3 elements` in subset S_1 .

Suppose we select `1 element` from the left side. Then we **MUST** select exactly `2 elements` from the right side because $1 + 2 = 3$.

Hence, `left[k]` must only combine with elements from `right[n-k]`.

11. Why Sort Right Side?

We later perform a binary search using `lower_bound()` on the collections in the right half. Binary search strictly requires a sorted array to operate correctly.

12. Binary Search Intuition

Suppose `leftSum = 2` and `totalSum = -6`.

We need $S_1 \approx \text{totalSum} / 2$ because the equation $\text{difference} = |2 \times S_1 - \text{totalSum}|$ reaches its minimum when S_1 gets closest to half of the total sum.

Compute our target for the right side:

$$\text{target} = (\text{totalSum} / 2) - \text{leftSum}$$

$$\text{target} = -3 - 2 = -5$$

Meaning: Find a `rightSum` that is as close as possible to `-5`.

13. Understanding `lower_bound()`

Suppose we have a sorted vector: `vec = {-11, -5, 2}`. Let our `target = -4`.

`lower_bound` returns an iterator to `2` because $2 \geq -4$ and it is the first such element that satisfies this condition.

Important

`lower_bound` **DOES NOT** return the closest element. It returns the **first element $\geq$ target**.

14. Why Check Previous Element?

Given `vec = {-11, -5, 2}` and `target = -4`:

- `lower_bound` matches: `2` → Distance: $|2 - (-4)| = 6$
- Previous element: `-5` → Distance: $|-5 - (-4)| = 1$

The previous element is a much better match. Therefore, the nearest element can either be `*it` or `*(it-1)`.

15. Why Only These Two Elements?

Sorted array property states that if $vec[k-1] < target \leq vec[k]$, then the closest element overall in the array must be either `vec[k]` or `vec[k-1]`. No other element can be closer.

16. Understanding These Conditions

Condition 1: `if(it != vec.end())`

Suppose `vec = {-11, -5, 2}` and `target = 100`.

There is no element ≥ 100 , so `it = vec.end()`.

```
-11  -5  2
      ^
    end()
```

Doing `*it` would access invalid memory. Hence, the check `if(it != vec.end())` acts as a critical safety check.

Condition 2: `if(it != vec.begin())`

Suppose `target = -20`. `lower_bound` returns an iterator to `-11`.

```
-11  -5  2
  ^
begin()
```

Now, performing `--it` would move the iterator before the first element, which is illegal. Therefore, checking `if(it != vec.begin())` prevents this memory issue.

18. Why We Check Both Candidates

We test both valid candidates:

- **Candidate 1:** `*it`
- **Candidate 2:** `*(it-1)`

For each, compute:

```
chosenSum = leftSum + rightSum;
```

Then update the global tracking answer:

```
answer = min(answer, abs(totalSum - 2 * chosenSum));
```

19. Complete Algorithm

1. **Step 1:** Split array into two halves: `left` and `right`.
2. **Step 2:** Generate all subset sums. Store them in `left[count]` and `right[count]`.
3. **Step 3:** Sort each vector inside `right[count]`.
4. **Step 4:** For every `leftSum` in `left[k]`, we look for a matching `rightSum` from `right[n-k]`.
5. **Step 5:** Compute the target value: $target = (totalSum / 2) - leftSum$.
6. **Step 6:** Execute `lower_bound()` on `right[n-k]` for the target.
7. **Step 7:** Validate and check both candidates: `it` and `it-1`.
8. **Step 8:** Compute `chosenSum = leftSum + rightSum`.
9. **Step 9:** Update the global minimum difference: `abs(totalSum - 2 * chosenSum)`.

Time Complexity

- **Subset Generation:** $O(2^n)$
- **Sorting:** $O(2^n \times \log(2^n))$
- **Searching:** $O(2^n \times \log(2^n))$
- **Overall Complexity:** $O(n \times 2^n)$ where $n \leq 15$, which is highly efficient.

Final Mental Model

Split array into two halves → Generate all subset sums grouped by subset size → For every left subset sum, find the best matching right subset sum using binary search → Combine them to form exactly n elements → Use `abs(totalSum - 2 * chosenSum)` to compute the minimum difference.

20. Complete Dry Run

Input

```
nums = [2, -1, 0, 4, -2, -9]
```

Length = 6 $\rightarrow 2n = 6 \Rightarrow n = 3$. We must choose exactly 3 elements for subset S_1 and the remaining 3 elements for subset S_2 .

Step 1: Calculate Total Sum

$totalSum = 2 + (-1) + 0 + 4 + (-2) + (-9) = -6$

Step 2: Split Array

```
left  = [2, -1, 0]
right = [4, -2, -9]
```


Step 3: Generate All Left Subset Sums

Mask	Subset Elements	Count	Sum	Stored Location
000	{}	0	0	<code>left[0] = {0}</code>
001	{2}	1	2	<code>left[1] = {2}</code>
010	{-1}	1	-1	<code>left[1] = {2, -1}</code>
011	{2, -1}	2	1	<code>left[2] = {1}</code>
100	{0}	1	0	<code>left[1] = {2, -1, 0}</code>
101	{2, 0}	2	2	<code>left[2] = {1, 2}</code>
110	{-1, 0}	2	-1	<code>left[2] = {1, 2, -1}</code>
111	{2, -1, 0}	3	1	<code>left[3] = {1}</code>

Final Left Table:

```
left[0] = {0}
left[1] = {2, -1, 0}
left[2] = {1, 2, -1}
left[3] = {1}
```

Step 4: Generate All Right Subset Sums

Mask	Subset Elements	Count	Sum	Stored Location
000	{}	0	0	<code>right[0] = {0}</code>
001	{4}	1	4	<code>right[1] = {4}</code>
010	{-2}	1	-2	<code>right[1] = {4, -2}</code>
011	{4, -2}	2	2	<code>right[2] = {2}</code>
100	{-9}	1	-9	<code>right[1] = {4, -2, -9}</code>
101	{4, -9}	2	-5	<code>right[2] = {2, -5}</code>
110	{-2, -9}	2	-11	<code>right[2] = {2, -5, -11}</code>
111	{4, -2, -9}	3	-7	<code>right[3] = {-7}</code>

Final Right Table (Before Sorting):

```
right[0] = {0}
right[1] = {4, -2, -9}
right[2] = {2, -5, -11}
right[3] = {-7}
```

Step 5: Sort Right Side

After sorting arrays inside each count index:

```
right[0] = {0}
right[1] = {-9, -2, 4}
right[2] = {-11, -5, 2}
right[3] = {-7}
```

Step 6: Start Searching

We need exactly $n = 3$ elements in total for S_1 .

Case 1: Take 0 elements from Left → Need 3 elements from Right.

- Left Sum = `0`
- Right Options from `right[3] = {-7}`
- Chosen Sum = $0 + (-7) = -7$
- Difference = $|-6 - 2 \times (-7)| = |-6 + 14| = 8$
- Current Answer = `8`

Case 2: Take 1 element from Left → Need 2 elements from Right.

- Select Left Sum = `2`
- Target = $(totalSum / 2) - leftSum = (-6 / 2) - 2 = -3 - 2 = -5$
- Search inside `right[2] = {-11, -5, 2}`
- `lower_bound(-5)` returns `-5` exactly.
- Chosen Sum = $2 + (-5) = -3$
- Difference = $|-6 - 2 \times (-3)| = |-6 + 6| = 0$
- Update Answer: `ans = 0`

Why Is This Perfect?

Chosen elements:

From Left: `{2}`

From Right: `{4, -9}`

Combined S_1 : `{2, 4, -9}` → Sum = `-3`

Remaining elements S_2 : `{-1, 0, -2}` → Sum = `-3`

Difference: $|-3 - (-3)| = 0$. Perfect partition found.

Optimal Partition Summary

- **Subset 1:** `[2, 4, -9]` → Sum = `-3`
- **Subset 2:** `[-1, 0, -2]` → Sum = `-3`
- **Final Difference:** `0`

Final Answer Output: `0`

Key Learnings From This Dry Run

We Never Explicitly Construct S_2

We only construct S_1 and compute the difference natively via $|totalSum - 2 \times S_1|$.

Why Binary Search Works

For every `leftSum`, we target $(totalSum / 2) - leftSum$. This precisely answers: "Find a `rightSum` that makes the final combined subset sum as close as possible to exactly half of the total sum."

Why We Check Both `it` and `it-1`

Because `lower_bound()` returns the first element $\leq$ target, not necessarily the closest numerical value. The absolute closest element could be the matched element itself or the item directly preceding it.

Why These Conditions Exist

- `if(it != vec.end())` — Prevents `*it` from reading garbage memory past the vector bounds.
- `if(it != vec.begin())` — Prevents `--it` from shifting backward before the first valid element.

Final Mental Picture

Split Array → Generate All Left Subset Sums & Right Subset Sums → Group By Count → Sort Right Side → For Every Left Sum, Find Best Right Sum Using Binary Search → Compute $|totalSum - 2 \times S_1|$ → Track Minimum Difference.